Copyright 2024–2026 Strawberry Creek Monitoring Group contributors.
Licensed under the Apache License, Version 2.0.
https://www.apache.org/licenses/LICENSE-2.0

The Mathematics and Design of FlameSkimmer

Technical Report

Daniel Boupha Christensen
Environmental, Health, and Safety, University of California, Berkeley
The Green Initiative Fund

October 13, 2026

Abstract

FlameSkimmer is the third detector built for Strawberry Creek, and its job is to know when it is out of its depth. DuskCrayfish gives a point forecast, and CobbleShoal adds a scale that says how noisy a sensor usually is, but neither can say that it has never seen conditions like the ones in front of it. A storm, a change of season or a long outage is forced into the same yes or no as an ordinary afternoon. FlameSkimmer keeps CobbleShoal's encoder unchanged and replaces its forecast head with a small Bayesian neural network, one that learns a distribution over its weights rather than a single value for each. Every forecast then carries two variances: the noise the sensor is expected to show, and the disagreement between plausible versions of the model. This document sets out the mathematics of that head, how it is trained, how its two variances feed the detection channels, and what the first trained checkpoint measured. The short version of the last part is that the machinery works and the uncertainty it reports does not yet respond to storms, for a reason that is now understood and can be fixed.

1 Introduction

A forecasting detector flags a reading when it lands somewhere the model did not expect. That works when the model's expectations are good. It goes wrong when the creek does something the model has rarely or never seen, because the model still produces a forecast, still produces a scale around it, and reports both with the same confidence it would have on a dry Tuesday in August. The resulting score cannot tell an unusual reading from an unusual situation.

Storms are the clearest case on this creek. Rain changes conductivity, stage and temperature together and quickly, rain-mode behaviour is unlabelled at three of the four nodes, and most of the corpus is dry. Uber met the same problem in traffic forecasting, where the departures that matter most, such as holidays, are exactly the periods with the least history behind them, and built a Bayesian network's uncertainty into anomaly detection for that reason [Zhu and Laptev, 2017]. Storms play the part of holidays here.

It helps to name the two kinds of uncertainty involved [Kendall and Gal, 2017]. *Aleatoric* uncertainty is noise in the data itself, the wobble a conductivity probe shows even when nothing is happening. More data does not reduce it. *Epistemic* uncertainty is uncertainty about the model, and it is large where training data was thin and small where it was plentiful. More data does reduce it. CobbleShoal already estimates the first through its heteroscedastic scale head. Nothing in the repository estimated the second before FlameSkimmer.

Table 1: The trained configuration of the first FlameSkimmer checkpoint. Values above the rule are shared with CobbleShoal's clock model; values below it belong to the Bayesian head and its training.

Setting	Value
Sites S, canonical variables V, nodes N	5, 5, 17
Window T	24 timesteps (6 hours at 15 min sampling)
Context features	2 (hour sine and cosine)
Hidden dimension d	64
Seed	20260806
Head layers	$128 \to 32 \to 32 \to 2$, tanh between
Prior	$\mathcal{N}(0, 1)$ on every head weight and bias
Initial ρ	-5, so $\sigma \approx 0.0067$
Supervised training targets N_{train}	141,534
KL warmup	linear over the first 10% of steps (752 steps)
Head samples at prediction S_{w}	64
Trainable parameters	72,637, of which 10,500 in the head
Operating false alarm rate q	10^{-3}
Calibrated threshold z_q	16.271

1.1 The trained configuration

The values in Table 1 are the settings of the first trained checkpoint, read from the model definition, the training script and the calibration artifact written beside the weights. The model is experimental and not served; these are the settings the evaluation in Section 6 measured.

The head is tiny next to the encoder, and that is on purpose. Its 5,250 weights and biases each carry a mean and a width, which is where the 10,500 comes from, and everything uncertain about the model lives in those numbers. The 62,137 parameters of the encoder are ordinary point estimates.

2 Data and Notation

2.1 Corpus and split

FlameSkimmer trains on exactly the windows CobbleShoal's clock model trained on. The corpus runs from 4 June 2025 to 29 April 2026, the span over which every site in the roster reports. The split is chronological, with validation starting on 1 February 2026, because a detector is deployed forward in time and adjacent 15 minute windows overlap by 23 of their 24 steps, so a shuffled split would put the training set into validation. Out of 19,096 fault-free anchors, 14,106 sit wholly on the training side and 4,990 on the validation side; the run used evenly spaced subsets of 12,000 and 3,000. The recorded spans are 4 June to 31 December 2025 for training and 25 February to 28 April 2026 for validation. The scaler is fitted on training rows only, and the two footbridge sensors the training side never observed, dissolved oxygen and floating conductivity, are held offline.

The corpus, the split, the anchors and the calibration routine are all imported from CobbleShoal's training script rather than copied. A change there reaches both models, and a comparison between them cannot be undone by the two copies drifting apart.

3 Model Architecture

3.1 The shared trunk

The encoder is CobbleShoal's NestedEncoder, unchanged: decay LSTM over each sensor node, sibling exchange within a site, directed transport downstream, and a final mix back into the nodes. Its mathematics is set out in the CobbleShoal report and is not repeated here.

What matters for FlameSkimmer is how it is built. The global random seed is set, the encoder is constructed, and then the seed is set again with a fixed offset of 10,000 before the head is constructed. CobbleShoal does the same. At the same seed the two encoders therefore start from byte-identical weights, which a unit test

Table 2: Notation. Symbols shared with the CobbleShoal report keep their meaning there.

Symbol	Meaning	Dimension
$\boldsymbol{h}$	encoder output for one node, node state and site context	$2d = 128$
a	last reading of the node, the anchor	scalar
w	one head weight or bias	scalar
μ_w, ρ_w	posterior mean and width parameter of w	scalar
σ_w	posterior standard deviation, $\log(1 + e^{\rho_w})$	scalar
s_p	prior standard deviation	scalar, 1
$q_\phi(w)$	approximate posterior over head weights	
β	KL weight during warmup	scalar, $0 \to 1$
μ_s, σ_s	forecast and scale under head sample s	N
$\bar{\mu}$	predictive mean	N
$v_{\text{al}}, v_{\text{ep}}$	aleatoric and epistemic variance	N
v_{tot}	total predictive variance	N
Λ	combined Fisher statistic	N

checks, and any difference between the two models' results belongs to their heads rather than to a luckier draw of the trunk. FlameSkimmer composes the encoder rather than inheriting CobbleShoal's base class, so none of the leave-one-out or reconstruction machinery comes with it.

3.2 The Bayesian head

The head reads the encoder output $\boldsymbol{h}$ for each node and produces two numbers, a forecast delta and a raw scale. It has three layers,

$$\boldsymbol{z}_1 = \tanh(W_1\boldsymbol{h} + \boldsymbol{b}_1), \qquad \boldsymbol{z}_2 = \tanh(W_2\boldsymbol{z}_1 + \boldsymbol{b}_2), \qquad (\Delta, r) = W_3\boldsymbol{z}_2 + \boldsymbol{b}_3, \tag{1}$$

and every entry of every W and $\boldsymbol{b}$ is a random variable rather than a number. The approximate posterior is mean-field Gaussian, so each weight is independent of the others,

$$q_\phi(w) = \mathcal{N}\big(\mu_w,\ \sigma_w^2\big), \qquad \sigma_w = \log\big(1 + e^{\rho_w}\big), \tag{2}$$

and the trainable parameters ϕ are the pairs (μ_w, ρ_w). The softplus keeps σ_w positive without a constraint, which is the parameterisation of Bayes by Backprop [Blundell et al., 2015] and of the bayesian-torch layers this head was modelled on [Krishnan et al., 2022]. The means start where an ordinary linear layer's weights start, uniform within $\pm 1/\sqrt{\text{fan in}}$, so the head begins at the same place CobbleShoal's scale head does. Every ρ_w starts at -5.

The forecast and its scale come out exactly as they do in CobbleShoal,

$$\mu = a + \Delta, \qquad \sigma = \text{softplus}(r) + 10^{-2}, \tag{3}$$

with the forecast anchored on the node's last reading and the scale floored so the likelihood cannot reward claiming perfect certainty.

3.3 Why only the head is Bayesian

Making the encoder Bayesian too would multiply the parameters that need a width, slow training, and force the whole encoder to be run once per sample at prediction time. The evidence says it would not buy much. Across four inference methods and eight datasets, networks with only some layers stochastic matched fully stochastic ones, with no systematic benefit from going further [Sharma et al., 2023]. Keeping the trunk deterministic means the encoder runs once per window and only the small head is sampled, so 64 samples cost 64 passes through about five thousand weights rather than 64 passes through the whole model.

3.4 Why two hidden layers

A single hidden layer would be cheaper, and it would be wrong in a way that matters here. Mean-field networks with one hidden layer provably cannot raise their uncertainty in the gap between two well separated regions of data [Foong et al., 2020]. Dry season and storm are, loosely, two such regions, and the point of the model is the uncertainty between and beyond them. The same work sees similar trouble empirically in deeper networks, which is why the deep ensemble reference in Section 7 exists.

3.5 Keeping the heteroscedastic output

The head still predicts a scale for every reading. Replacing the per-reading scale with weight uncertainty alone would lose the aleatoric part, and the dispersion channel in Section 5.4 needs exactly that part. Combining an input-dependent noise scale with weight uncertainty in one model is the construction of Kendall and Gal [2017]. This is also why two off-the-shelf alternatives were set aside: post hoc Laplace approximation and variational Bayesian last layers, as implemented in their libraries, both learn a single noise level shared by every input, which throws away the per-reading scale.

4 Training

4.1 The evidence lower bound

Training fits ϕ by maximising the evidence lower bound,

$$\mathcal{L}_{\mathrm{ELBO}}(\phi) = \mathbb{E}_{q_\phi(w)}\big[\log p(\mathcal{D} \mid w)\big] - \mathrm{KL}\big(q_\phi(w) \parallel p(w)\big), \tag{4}$$

which trades fitting the data against staying close to the prior. The first term rewards weights that explain the readings. The second charges for every bit of certainty the posterior claims beyond what the prior allows, so a weight the data says nothing about drifts back toward $\mathcal{N}(0, s_p^2)$, and its width grows, which is where epistemic uncertainty comes from.

The likelihood is CobbleShoal's masked Gaussian negative log likelihood,

$$\mathcal{L}_{\mathrm{nll}} = \frac{1}{|M|} \sum_{i \in M} \left[\tfrac{1}{2} \left(\frac{y_i - \mu_i}{\sigma_i} \right)^2 + \log \sigma_i \right], \tag{5}$$

averaged over the entries M where a target was observed and the anchor was fresh enough to predict a delta from.

4.2 The KL term, summed

With a Gaussian posterior and a Gaussian prior the divergence has a closed form per weight,

$$\mathrm{KL}\big(\mathcal{N}(\mu_w, \sigma_w^2) \parallel \mathcal{N}(0, s_p^2)\big) = \log \frac{s_p}{\sigma_w} + \frac{\sigma_w^2 + \mu_w^2}{2 s_p^2} - \frac{1}{2}, \tag{6}$$

and the KL of the whole head is the sum of this over all 5,250 weights and biases. The sum matters. The bayesian-torch base layer returns the mean over weights instead, which shrinks the penalty by a factor of the parameter count and quietly turns the ELBO into something else. The same library also fills its prior width with an argument named `prior_variance`, so what it calls a variance is used as a standard deviation. FlameSkimmer writes its own small layer instead of depending on that library, which avoids both, and avoids one more import paying the network file system cost on the OCF host. A unit test checks the summed KL against PyTorch's own `kl_divergence`, and another checks that it comes out at zero when the posterior equals the prior.

4.3 Scaling and warmup

The likelihood above is a mean over targets, so the KL has to be put on the same per-target footing. The loss for one minibatch is

$$\mathcal{L} = \mathcal{L}_{\mathrm{nll}} + \beta \, \frac{\mathrm{KL}(q_\phi \,\|\, p)}{N_{\mathrm{train}}}, \tag{7}$$

where $N_{\mathrm{train}} = 141{,}534$ is the number of supervised (anchor, node) targets in the training set, counted with the same mask the likelihood uses. Every minibatch carries the same share of the KL. Blundell's paper also proposes a schedule that weights the KL by $2^{M-i}/(2^M - 1)$ for batch i of M, and the BLiTZ library implements it [Esposito, 2020], but that schedule puts nearly all of the KL on the first batch of each epoch and almost none on the rest, so it is not used.

The weight β climbs linearly from 0 to 1 over the first 10% of optimiser steps and then stays at 1. Starting at zero lets the head learn a sensible forecast before the prior starts pulling on it, and holding at 1 afterwards means the trained model optimises the true ELBO, with no tempering.

Table 3: Training history. NLL is the training likelihood term, KL/N the scaled divergence, and the last two columns the mean epistemic share on 600 training and 600 validation anchors. Epoch 5 was kept.

Epoch	NLL	KL/N	β	Val NLL	Share train	Share val
1	−1.699	0.166	0.25	−1.933	0.160	0.147
2	−2.142	0.162	0.50	−2.110	0.095	0.080
3	−2.332	0.159	0.75	−2.264	0.155	0.126
4	−2.215	0.156	1.00	−2.304	0.131	0.122
5	−2.333	0.152	1.00	−2.315	0.135	0.115
6	−2.402	0.148	1.00	−2.173	0.076	0.058
7	−2.431	0.145	1.00	−2.254	0.123	0.091
8	−2.330	0.142	1.00	−2.182	0.124	0.097
9	−2.548	0.140	1.00	−2.298	0.101	0.071
10	−2.652	0.139	1.00	−2.205	0.129	0.099
11	−2.545	0.137	1.00	−2.182	0.108	0.078

4.4 Local reparameterization

The expectation in the ELBO is estimated by sampling. The obvious way is to draw one weight matrix and push the whole minibatch through it, but then every window in the batch shares the same noise, and the gradient estimate does not get better as the batch grows. The local reparameterization trick [Kingma et al., 2015] samples the layer's output instead. For an input $\boldsymbol{x}$ to a layer with independent Gaussian weights, each pre-activation is itself Gaussian,

$$b_j \sim \mathcal{N}\left(\sum_i x_i\,\mu_{ji} + \mu_{c_j},\quad \sum_i x_i^2\,\sigma_{ji}^2 + \sigma_{c_j}^2\right), \tag{8}$$

so one standard normal draw per output element gives an exact sample, with independent noise for every window. The gradient variance then falls with batch size as it should. The training forward uses this at every layer.

4.5 Optimiser and stopping

Training uses Adam at a learning rate of 10^{-3}, batches of 64 anchors, gradient clipping at 1.0, and CobbleShoal's stopping rule: the learning rate halves after two epochs without improvement, and training stops after six, keeping the best epoch. The quantity watched is the validation negative ELBO at $\beta = 1$, with a fixed noise generator so the change between epochs is the weights and not the draw. Table 3 shows the run.

The KL term never came close to dominating the loss, which was the first thing to rule out. Its scaled size of about 0.15 sits beside a likelihood of about −2.3.

5 Prediction and Detection

5.1 The predictive distribution

At prediction time the encoder runs once and the head runs once for each of $S_\mathrm{w} = 64$ sampled weight sets, giving a forecast μ_s and a scale σ_s per sample. The predictive distribution is a mixture of 64 Gaussians, and it is summarised by its first two moments,

$$\bar{\mu} = \frac{1}{S_\mathrm{w}} \sum_s \mu_s, \tag{9}$$

$$v_\mathrm{al} = \frac{1}{S_\mathrm{w}} \sum_s \sigma_s^2, \qquad v_\mathrm{ep} = \frac{1}{S_\mathrm{w}} \sum_s \left(\mu_s - \bar{\mu}\right)^2, \qquad v_\mathrm{tot} = v_\mathrm{al} + v_\mathrm{ep}. \tag{10}$$

This is the law of total variance. The aleatoric part is the noise each version of the model expects, averaged; the epistemic part is how much the versions disagree about the forecast itself. The variance is the population form, dividing by S_w, because that is what the moment-matched mixture uses. The ratio $v_\mathrm{ep}/v_\mathrm{tot}$, the *epistemic share*, is the number the evaluation watches.

5.2 Sampling whole weights at prediction time

Training samples outputs; prediction samples whole weight matrices, and the difference is deliberate. Local reparameterization gives every window in a batch its own independent noise, which is what training wants and what scoring cannot have: a window's score would then depend on which other windows happened to share its batch and where it sat in it. Drawing S_{w} complete weight sets and passing every window through the same ones removes that. For any single window the two schemes give exactly the same distribution, since each layer's weights are used once per window and are independent of the layers before them. A unit test checks that a window scores identically alone and stacked beside another.

5.3 Fixed serving noise

The 64 weight sets are drawn once, when the model is constructed, from a fixed seed and a private generator, and reused on every call. Two runs over the same window therefore give the same score to the last bit, which matters because an alert that appears on one run and vanishes on the next is worse than either answer. Drawing from a private generator leaves the global random state untouched, so constructing the model does not disturb anything seeded after it. What is stored is the standard normal noise rather than the weights themselves, so the same draw stays valid however training later moves μ and ρ.

5.4 Two channels, and which variance each reads

FlameSkimmer has the two channels that come free with a forecast head, and the rule that matters most in the whole design is which variance each one reads.

Excursion is the standardised one-step residual against the *total* predictive standard deviation,

$$c_{\mathrm{exc}} = \frac{|y - \bar{\mu}|}{\sqrt{v_{\mathrm{tot}}}}. \tag{11}$$

A reading far from the forecast in conditions the model knows well should count for a lot. The same distance in conditions the model admits it does not know should count for less, and dividing by the total variance does exactly that.

Dispersion reads the *aleatoric* part only,

$$c_{\mathrm{disp}} = \max\!\left(0,\ \log \frac{\max(\sqrt{v_{\mathrm{al}}}, \varphi)}{\max(\hat{s}, \varphi)}\right), \tag{12}$$

where $\hat{s}$ is the observed spread over the last 8 steps and $\varphi = 10^{-3}$ is a floor. Dispersion asks whether a sensor is flatter than its own noise, which is how a frozen probe gives itself away. If the epistemic variance were included, the expected spread would swell whenever the model was unsure, an ordinary sensor would then look too flat by comparison, and every storm would read as a stuck sensor. The functions computing both channels are CobbleShoal's frozen ones, called with different scales; their arithmetic is not touched, because the calibrated artifacts depend on it.

5.5 Nulls, combination and threshold

Everything after the channels is CobbleShoal's procedure, run on FlameSkimmer's scores. Each channel becomes a right-tail probability against its own fault-free distribution, empirical below the 95th percentile and a fitted Generalised Pareto above it [Pickands III, 1975]. The two probabilities are combined per node as $\Lambda = -2\sum_j \log p_j$ [Fisher, 1925], and the threshold comes from peaks over threshold on the fault-free values of Λ [Siffer et al., 2017],

$$z_q = u - \sigma \log\!\left(\frac{q\,n}{N_u}\right) \quad (\gamma = 0). \tag{13}$$

The fit used 400 fault-free training windows, giving $n = 4{,}769$ node scores, an initial threshold $u = 10.135$ with $N_u = 96$ exceedances, $\sigma = 2.044$ and $\gamma = 0$, so $z_q = 16.271$ at $q = 10^{-3}$. Measured on 2,118 held-out validation scores the exceedance rate was 4.7×10^{-4}.

Rain is handled outside the model, as it is for CobbleShoal. The rain gate moves the threshold and never the score, and FlameSkimmer declares no built-in support behaviour. The comparison in the next section uses the ungated threshold for both models.

Table 4: Forecast quality on validation, FlameSkimmer against CobbleShoal. Lower NLL and CRPS are better; the 90% interval should cover close to 90%. Footbridge has only 21 supervised targets on the validation side.

		NLL		CRPS		Coverage 90	
Node	n	Flame	Cobble	Flame	Cobble	Flame	Cobble
all nodes	31,650	−1.460	−1.157	0.0335	0.0330	0.972	0.961
north_fork_0 cond.	2,996	−1.588	−1.410	0.0243	0.0277	0.972	0.977
north_fork_0 depth	2,996	−0.503	0.461	0.0470	0.0522	0.977	0.988
north_fork_0 temp.	2,996	−1.432	0.275	0.0178	0.0154	0.983	0.894
footbridge cond.	21	−1.222	0.045	0.0383	0.1041	1.000	1.000
footbridge depth	21	−1.052	−0.590	0.0490	0.0650	0.952	1.000
footbridge temp.	21	−1.712	−1.443	0.0220	0.0252	1.000	1.000
south_fork_1 cond.	1,536	−1.382	−0.681	0.0330	0.0543	0.958	0.980
south_fork_1 depth	1,536	−0.540	−0.530	0.0701	0.0688	0.993	0.991
south_fork_1 temp.	1,536	−1.645	−1.755	0.0235	0.0206	0.990	0.978
south_fork_2 cond.	2,997	−1.924	−1.664	0.0217	0.0258	0.973	0.939
south_fork_2 depth	2,997	−0.903	−1.030	0.0733	0.0496	0.989	0.985
south_fork_2 temp.	2,997	−2.115	−2.336	0.0155	0.0125	0.960	0.934
oxford cond.	3,000	−1.980	−1.894	0.0157	0.0166	0.984	0.985
oxford depth	3,000	−0.972	−0.899	0.0570	0.0590	0.907	0.968
oxford temp.	3,000	−2.141	−2.184	0.0160	0.0148	0.985	0.943

6 Evaluation

FlameSkimmer was compared with CobbleShoal's clock model, trained by the same script on the same split from the same seed. Every number is in the scaler's standardised units, and the evaluation checks that both models were scaled identically before comparing them. Results are reported per node; a mean across nodes hides the nodes where a model is worse.

6.1 Forecast quality

Table 4 gives the negative log likelihood, the continuous ranked probability score and the coverage of the central 90% interval on the 3,000 validation anchors. The CRPS uses the closed form for a Gaussian [Gneiting and Raftery, 2007], applied to the moment-matched predictive distribution, and was copied from uncertainty-toolbox [Chung et al., 2021] rather than adding the package. The NLL here includes the $\frac{1}{2}\log 2\pi$ constant the training loss leaves out and is taken over the full predictive distribution, so it is not directly comparable with the validation NLL in Table 3.

The picture is mixed. FlameSkimmer is better on likelihood overall and better on both NLL and CRPS at every conductivity node, which is the variable that is scored for detection. It is worse on NLL at four nodes and on CRPS at six, mostly temperature and stage on the south fork and at oxford, so the proposed bar of no node worse on either measure is not met. Its intervals are also too wide. The central 90% interval covers 97.2% of validation targets, outside the 85 to 95% band that would count as honest. Wide intervals improve the likelihood on rough nodes and cost CRPS on smooth ones, which fits the pattern in the table.

6.2 Epistemic behaviour

The test the model exists to pass is whether its epistemic share rises where it has seen little. A step counts as rain when more than 0.1 mm fell over the preceding six hours, from the Open-Meteo archive. Table 5 gives the share on every supervised target, split by side and by rain, along with the median epistemic standard deviation in absolute terms.

The share falls in rain rather than rising, on both sides of the split and in all three event windows. The last column says why. The absolute epistemic standard deviation is 0.0149 or 0.0147 in every group, so the epistemic variance is effectively a constant. What moves in rain is the aleatoric part: the scale head correctly predicts a noisier sensor, the total grows, and a constant divided by a larger total gets smaller. The model is not saying it is more certain in storms. It is not saying anything with its epistemic variance at all.

Table 5: Epistemic share $v_{\text{ep}}/v_{\text{tot}}$ per supervised target, and the median epistemic standard deviation $\sqrt{v_{\text{ep}}}$. The three event rows count rain steps only; the January event is the botanical garden actuator.

Group	n	Median share	Quartiles	Median $\sqrt{v_{\text{ep}}}$
Training, dry	131,459	0.132	0.082 to 0.215	0.0149
Training, rain	10,075	0.066	0.008 to 0.151	0.0147
Validation, dry	29,385	0.132	0.049 to 0.184	0.0149
Validation, rain	2,265	0.027	0.002 to 0.117	0.0147
Event, November 2025 rain	813	0.021	0.001 to 0.084	
Event, April 2026 rainfall	810	0.064	0.003 to 0.157	
Event, January 2026 actuator	5,664	0.022	0.002 to 0.092	

Table 6: Peak score at the graded node in each labelled event window. Entries labelled relative only are reported, not graded.

Event	Site	Flame	Cobble	Label
June 2025 discharge	south_fork_1	17.655	34.539	anomaly, both flagged
September 2025 overnight	south_fork_1	23.740	47.566	anomaly, both flagged
September 2025 overnight	south_fork_2	15.796	28.405	anomaly, both missed
March 2026 hydrant	north_fork_0	19.467	46.026	anomaly, both flagged
June 2025 discharge	south_fork_2	17.880	26.872	relative only
August 2025 sprinklers	north_fork_0	11.228	13.507	relative only
November 2025 foam	north_fork_0	23.387	48.388	relative only

6.3 Detection

Table 6 scores every labelled event in the catalogue. For each entry the peak combined score at the named site's conductivity node, over every anchor in the event window, is compared with each model's threshold at $q = 10^{-3}$: 16.271 for FlameSkimmer and 29.070 for CobbleShoal. The two thresholds are on different scales, since one combines two channels and the other four.

Both models flag three of the four labelled anomalies and both miss the south_fork_2 half of the September event, FlameSkimmer by about 3% of its threshold. The margins are thinner for FlameSkimmer, though. Its peaks sit 8 to 46% above its threshold, where CobbleShoal's sit 19 to 64% above its own, which is what dropping the leave-one-out and reconstruction channels would be expected to cost.

6.4 False alarm rate

Table 7 gives the rate at which fault-free validation node scores crossed each model's threshold at $q = 10^{-3}$, overall and split by rain.

The overall rate is close to nominal for both models, well inside the order of magnitude that the peaks over threshold diagnostics already treat as acceptable. In rain FlameSkimmer flags at about half CobbleShoal's rate. That looks like the payoff the model was built for, but Section 6.2 rules out the intended explanation, since the epistemic variance does not change in rain. The likelier cause is the wider total intervals of Section 6.1, which damp the excursion channel everywhere and matter most where readings move most.

6.5 The gate, and why it failed

The plan set one test for whether the Bayesian head is doing its job: the median epistemic share on validation rain steps should be at least 1.5 times the median on training dry steps. It is $0.027/0.132 = 0.20$ times.

The cause is in the trained widths. Table 8 gives the posterior standard deviations after training. Every one of them started at $\log(1 + e^{-5}) \approx 0.0067$, and they have barely moved.

The ceiling on the first layer is not a coincidence. Adam moves each parameter by roughly the learning rate per step at most, and the kept epoch came after about 940 steps at a learning rate of 10^{-3}. Starting from -5, no ρ_w could have climbed much past -4.06, which is $\sigma_w \approx 0.017$. The largest width actually learned is

Table 7: Empirical false alarm rate on fault-free validation anchors at $q = 10^{-3}$. CobbleShoal has more finite scores because its leave-one-out channel does not need a fresh anchor.

	All		Dry		Rain	
Model	Rate	n	Rate	n	Rate	n
FlameSkimmer	1.07×10^{-3}	31,722	7.5×10^{-4}	29,445	5.3×10^{-3}	2,277
CobbleShoal	1.36×10^{-3}	37,626	6.9×10^{-4}	34,905	9.9×10^{-3}	2,721

Table 8: Posterior standard deviations of the head after training. Every value started at 0.0067.

Layer	Median σ, weights	Largest σ, weights	Median σ, biases
$128 \to 32$	0.0129	0.0160	0.0093
$32 \to 32$	0.0064	0.0108	0.0061
$32 \to 2$	0.0053	0.0068	0.0036

0.016. The widths were limited by the optimiser and the starting point, not chosen by the data, and early stopping kept the epoch immediately after β reached 1, before the prior had any time at full strength.

That also explains why the epistemic variance is constant. When every width sits near the same small value, the variance the head adds is roughly that width squared times the squared size of the activations feeding each layer, and after a tanh those activations have much the same size for almost every input. The variance that results barely depends on the window, so it cannot rise in a storm.

None of this says that a Bayesian head cannot do the job. It says this one was not trained long enough, or started too narrow, for its widths to learn anything.

7 Limitations and Future Work

The first thing to fix is the training of the widths. Three changes address Section 6.5 directly: start ρ less negative, for example at -3, which gives $\sigma \approx 0.049$; give the ρ parameters their own, larger learning rate; and stop early only after β has been at 1 for several epochs. Each is a change to a default rather than to the design, and the gate should be measured again after them before anything else is built on the model.

The plan also calls for a reference when the gate fails: a five-seed deep ensemble of the same trunk with CobbleShoal's deterministic scale head, reported beside FlameSkimmer. Ensembles match or beat approximate Bayesian networks in most comparisons [Lakshminarayanan et al., 2017] and held up best under dataset shift in the largest benchmark of the question [Ovadia et al., 2019], though that benchmark is classification. The ensemble is not Bayesian over weights, but it is the bar a Bayesian head should clear, and it has not yet been run. Monte Carlo dropout, the other cheap option, was the weakest of four methods in a rainfall-runoff LSTM benchmark [Klotz et al., 2022] and is not planned.

The chronological split was expected to put most of the rain on the validation side. By anchor count it does not: 7.0% of training anchors and 7.3% of validation anchors had rain in the preceding six hours. The storms may still differ in size or season between the two sides, but the split is a weaker test of shift than it was taken to be, and a blocked split or a held-out storm would make a sharper one.

Only one seed has been trained. Every result in Section 6 should be repeated across several before it is relied on, and reported per seed.

The event catalogue has no true negatives, no storm that the group has labelled as a known non-event. The rain false alarm comparison therefore rests on fault-free statistics alone. One labelled storm would turn it into a real case.

FlameSkimmer is not served. A serving detector, its entry in the serving registry and updates to the README and user manual follow only once the evaluation is accepted. Whether the epistemic signal should ever become an alert type of its own is a separate decision, and nothing in this design adds an uncertainty rule to the alerting path.

Finally, every number in this document describes one experimental checkpoint and the calibration fitted alongside it. They will move when the model is retrained, and some of them, by design, should.

References

Lingxue Zhu and Nikolay Laptev. Deep and confident prediction for time series at Uber. In *IEEE International Conference on Data Mining Workshops (ICDMW)*, 2017. URL https://arxiv.org/abs/1709.01907.

Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. URL https://arxiv.org/abs/1703.04977.

Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1613–1622, 2015. URL https://arxiv.org/abs/1505.05424.

Ranganath Krishnan, Pi Esposito, and Mahesh Subedar. Bayesian-torch: Bayesian neural network layers for uncertainty estimation. https://github.com/IntelLabs/bayesian-torch, 2022. Read at commit aa7e57b.

Mrinank Sharma, Sebastian Farquhar, Eric Nalisnick, and Tom Rainforth. Do Bayesian neural networks need to be fully stochastic? In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 206 of *Proceedings of Machine Learning Research*, 2023. URL https://proceedings.mlr.press/v206/sharma23a.html.

Andrew Y. K. Foong, David R. Burt, Yingzhen Li, and Richard E. Turner. On the expressiveness of approximate inference in Bayesian neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL https://arxiv.org/abs/1909.00719.

Piero Esposito. BLiTZ: Bayesian layers in torch zoo. https://github.com/piEsposito/blitz-bayesian-deep-learning, 2020. Read at commit 5af1174.

Diederik P. Kingma, Tim Salimans, and Max Welling. Variational dropout and the local reparameterization trick. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015. URL https://arxiv.org/abs/1506.02557.

James Pickands III. Statistical inference using extreme order statistics. *The Annals of Statistics*, 3(1): 119–131, 1975. doi: 10.1214/aos/1176343003.

Ronald A. Fisher. *Statistical Methods for Research Workers*. Oliver and Boyd, Edinburgh, 1925.

Alban Siffer, Pierre-Alain Fouque, Alexandre Termier, and Christine Largouët. Anomaly detection in streams with extreme value theory. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1067–1075, 2017. doi: 10.1145/3097983.3098144.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

Youngseog Chung, Ian Char, Han Guo, Jeff Schneider, and Willie Neiswanger. Uncertainty toolbox: an open-source library for assessing, visualizing, and improving uncertainty quantification, 2021. URL https://arxiv.org/abs/2109.10254.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. URL https://arxiv.org/abs/1612.01474.

Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1906.02530.

Daniel Klotz, Frederik Kratzert, Martin Gauch, Alden Keefe Sampson, Johannes Brandstetter, Günter Klambauer, Sepp Hochreiter, and Grey Nearing. Uncertainty estimation with deep learning for rainfall–runoff modeling. *Hydrology and Earth System Sciences*, 26:1673–1693, 2022. doi: 10.5194/hess-26-1673-2022.